



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor

A Report

Presented to the University of Dublin, Trinity College
in partial fulfilment of the requirements for the degree of

BA(Mod) in Science in Computer Science

Supervisor: Stephen Farrell

May 2026

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor, BA(Mod) in Science in Computer Science
University of Dublin, Trinity College, 2026

Supervisor: Stephen Farrell

This project explores the development of a local test environment for evaluating Encrypted Client Hello (ECH) using the Lighttpd web server. ECH, an extension to TLS 1.3, encrypts sensitive information in the ClientHello message to improve user privacy. The goal of this report is to build, configure, and test a working Lighttpd installation capable of performing ECH handshakes using BoringSSL and to verify its functionality through practical experimentation. The work includes compiling Lighttpd with ECH-capable TLS libraries, generating ECH configuration files, and validating results with tools such as `bssl client`, `curl`, Wireshark, and modern browsers. The outcome demonstrates the feasibility of integrating ECH within lightweight web server environments and documents configuration issues, debugging approaches, and performance observations.

Acknowledgments

Thank you Mum & Dad.

SEÁN JOSEPH LAWLOR

University of Dublin, Trinity College
May 2026

Contents

Abstract	i
Acknowledgments	ii
Chapter 1 Introduction	1
1.1 Motivation	1
1.2 Project Objectives	2
1.3 Research Contributions	4
Chapter 2 State of the Art	5
2.1 Background	5
2.1.1 Transport Layer Security 1.3	5
2.1.2 Encrypted ClientHello	5
2.1.3 TLS Libraries Supporting ECH	5
2.1.4 Lighttpd	5
2.1.5 Headless browsers	5
2.2 Closely-Related Work	5
2.3 Summary	5
Chapter 3 Design	7
3.1 Problem Formulation	7
3.1.1 Identified Challenges	7
3.1.2 Proposed Work	7
3.2 Overview of the Design	8
3.3 Summary	8
Chapter 4 Implementation	9
4.1 Overview of the Solution	9
4.2 Component One	9
4.3 Summary	10

Chapter 5	Evaluation	11
5.1	Experiments	11
5.2	Results	12
5.3	Summary	13
Chapter 6	Conclusions & Future Work	14
6.1	Future Work	14
Bibliography		15

List of Tables

5.1	Experimental matrix used for ECH behavioural validation	11
-----	---	----

List of Figures

Chapter 1

Introduction

Encrypted ClientHello (ECH) is an emerging extension to the TLS 1.3 protocol which aims to protect sensitive metadata during the TLS handshake. While TLS encrypts application data, the initial ClientHello message remains unencrypted, revealing information such as the Server Name Indication (SNI) and Application-Layer Protocol Negotiation (ALPN). This information being shown in plain-text allows bad actors to monitor, profile, censor, or intercept users, even when TLS encryption is otherwise secure. ECH encrypts the ClientHelloInner message using Hybrid Public Key Encryption (HPKE) which conceals the intended domain and strengthens privacy at the network layer.

Lighttpd is a lightweight and high-performance open-source web server designed for environments where efficiency, low memory usage, and simplicity are essential. Unlike larger server platforms such as Apache or Nginx, Lighttpd prioritises a minimal footprint while maintaining support for modern web standards and extensible features through its modular architecture. Its event-driven design allows it to handle large volumes of concurrent connections with reduced overhead, making it suitable for embedded systems, resource-constrained deployments, and experimental research setups.

As ECH is relatively new and still undergoing standardisation, ECH is poorly tested. This project focuses on building a test environment. For ECH using a web server built with Lighttpd. The tests will include

1.1 Motivation

Metadata leakage remains a critical weakness even when content is fully encrypted. The SNI field in TLS has become a significant point of exposure, enabling censorship regimes, tracking systems, and other bad actors to monitor user activity. For organisations committed to user privacy including civil society groups, investigative journalists, and human rights organisations mitigating SNI-based surveillance is essential.

ECH offers a promising defence, but its deployment is not straightforward. Its reliance on experimental TLS libraries, DNS infrastructure, and non-standardised tooling creates high barriers for developers and researchers. There is also limited publicly available guidance documenting how ECH-enabled servers should be built, configured, and tested.

The motivation for this project is therefore twofold:

- To create a reproducible ECH testing environment suitable for experimentation, debugging, and educational use.
- To evaluate Lighttpd’s experimental ECH implementation, documenting its behaviour, limitations, and compatibility with BoringSSL’s tooling.

A reliable ECH test harness lowers the barrier to future deployment, facilitates research into evolving protocol behaviour, and supports wider adoption.

1.2 Project Objectives

The central research question guiding this work is:

How can Encrypted ClientHello be enabled, tested, and validated on a Lighttpd web server in an efficient reproducible manner?

This question is explored through the following objectives:

- **Understand the ECH protocol and surrounding technologies.** This involves studying TLS 1.3, HPKE, DNS integration requirements, and the structure of the ClientHelloInner and ClientHelloOuter messages. Practical experiments with BoringSSL are used to deepen understanding.
- **Compile and configure Lighttpd with ECH support.** Lighttpd must be rebuilt from source and linked against BoringSSL. This includes modifying build flags, resolving dependency issues, and integrating ECH key material into the server configuration.
- **Develop an automated ECH test harness.** Initial testing uses the `bssl client` debugging tool to inspect handshake behaviour. Later work incorporates automated browser-based testing with Playwright to simulate realistic client interactions.
- **Analyse behaviour through logged TLS handshakes.** Focus is placed on interpreting negotiation states, debugging errors such as `TLSV1_ALERT_ECH_REQUIRED`, and determining whether the server accepts, rejects, or attempts ECH negotiation under various configurations.

- **Incorporate automated browser-based testing.** Playwright and headless browsers are used to simulate realistic client behaviour, validate ECH negotiation in full browser stacks, and provide reproducible interaction traces for analysis.

These objectives establish the technical foundation for the implementation and analysis conducted throughout the project.

Period	Goals / Deliverables
Oct 14–27	- Begin learning LaTeX basics - Set up Ubuntu, build tools, Git
Oct 28–Nov 10	- Study ECH protocol - Test basic TLS setup and certificates - Research/testing strategy - Document initial setup in LaTeX
Nov 11–24	- Enable ECH on Lighttpd - Capture test results with <code>bssl</code> / <code>openssl</code> - Upload results to GitHub
Nov 25–Dec 8	- Test ECH using Chrome, Firefox, etc. - Introduce Playwright for automated headless testing - Validate ECH behaviour in real browsers - Update LaTeX documentation
Dec 9–22	- Basic performance measurements - Expand Playwright tests (timing, errors) - Begin structured LaTeX report
Dec 23–Jan 5	- Mid-project review and summary - Update GitHub with configs and Playwright code - Produce short progress PDF
Jan 6–19	- Refine ECH setup and tests - Improve Playwright traces, logs, screenshots - Fix server or test issues
Jan 20–Feb 2	- Finalise ECH implementation on Lighttpd - Automate recurring Playwright tests (CI-ready)
Feb 3–16	- Run comprehensive ECH tests - Compare <code>bssl client</code> vs Playwright behaviour - Draft full LaTeX report
Feb 17–Mar 1	- Review and refine LaTeX report - Add figures, tables, diagrams, browser traces
Mar 2–15	- Supervisor feedback and revisions - Finalise Playwright scripts and documentation
Mar 16–29	- Prepare presentation/demo - Test demo setup (Lighttpd + Playwright interaction)
Mar 30–Apr 12	- Submit final report and project deliverables - Stretch Goal: PR to Lighttpd - Present project

A biweekly timeline is shown below fig 1.2.1

1.3 Research Contributions

This work makes the following contributions:

- **A reproducible environment for building and testing ECH on Lighttpd.** This includes the full toolchain for compiling BoringSSL, integrating it with Lighttpd, and generating ECH key material.
- **A detailed examination of ECH handshake behaviour.** The project documents negotiation states, error modes, and observable behaviours such as `ECH_REJECTED` that arise during experimental testing.
- **Analysis of Lighttpd’s experimental ECH implementation.** This includes identifying limitations, configuration pitfalls, and mismatches between BoringSSL’s ECH formats and Lighttpd’s expectations.
- **Platwright and headless browser.** The project establishes tests using playwright and headless browser

The methodology developed here supports future researchers studying ECH adoption, debugging server implementations, and building larger interoperability test suites.

Chapter 2

State of the Art

This chapter provides an overview of the technologies, standards, and prior work relevant to the development of an Encrypted ClientHello (ECH) test setup for Lighttpd. It first introduces the technical background required to understand ECH within the TLS 1.3 protocol and then surveys work related to ECH implementations, testing methodologies, and deployment models. The chapter concludes with a summary that compares existing approaches and highlights the gap addressed by this dissertation.

2.1 Background

2.1.1 Transport Layer Security 1.3

2.1.2 Encrypted ClientHello

2.1.3 TLS Libraries Supporting ECH

2.1.4 Lighttpd

2.1.5 Headless browsers

2.2 Closely-Related Work

2.3 Summary

This chapter introduced the technical foundations of TLS 1.3 and ECH, reviewed existing implementations of ECH across various libraries, and discussed prior work in deployment, testing, and analysis. Existing research demonstrates both the potential and challenges of deploying ECH in real-world environments.

However, very limited work has examined ECH integration on lightweight web servers such as Lighttpd. Most existing literature focuses on NGINX or general-purpose TLS libraries, leaving a gap in the evaluation of smaller, embedded-friendly servers. This dissertation addresses this gap by building a reproducible ECH testing environment specifically for Lighttpd.

Chapter 3

Design

At the beginning of each chapter, a description should introduce the reader to the content of the chapter. The description should explain to the reader the layout of the chapter, the contribution that the chapter makes to the overall dissertation and the contribution of the individual sections towards the overall chapter.

3.1 Problem Formulation

This section should provide the reader with an overall description of the problem that will be addressed in the dissertation. In contrast to a generic discussion of the dissertation topic in the Introduction chapter, this section should provide a detailed discussion of the problem that has been identified based on the existing work that has been discussed in the preceding chapter.

In some dissertations, it may make sense to convert this section into a short chapter of its own which follows the discussion of the existing work and precedes the discussion of the work of the dissertation.

3.1.1 Identified Challenges

This section should present a short description of the gaps in the existing work and the relationship of these gaps to the work described in this dissertation.

3.1.2 Proposed Work

This section should provide a thorough description of the problem and an overview of the work proposed to address the problem.

3.2 Overview of the Design

A description of the approach that addresses the problem identified above.

3.3 Summary

Every chapter aside from the first and last chapter should conclude with a summary.

Chapter 4

Implementation

Guess what? At the beginning of each chapter, a description should introduce the reader to the content of the chapter. The description should explain to the reader the layout of the chapter, the contribution that the chapter makes to the overall dissertation and the contribution of the individual sections towards the overall chapter.

4.1 Overview of the Solution

```
x = 1
if x == 1:
    # indented four spaces
    print("x is 1.")
```

Listing 4.1: Lengthy caption explaining the code to the reader

The code in listing 4.1 is a demonstration how to include a file with code into the template.

4.2 Component One

The code in listing 4.2 is a demonstration how to include code in the template.

```
x = 1
if x == 1:
    # indented four spaces
    print("x is 1.")
```

Listing 4.2: Second Lengthy caption

4.3 Summary

Every chapter aside from the first and last chapter should conclude with a summary.

Chapter 5

Evaluation

The Evaluation chapter should present a comparison of the work that forms the basis of the dissertation and existing work. At a higher level, it should demonstrate an awareness of the relationship of the dissertation work to the research area that it is based in.

5.1 Experiments

To evaluate the correctness and behavioural properties of the implemented ECH test harness, a full $2 \times 2 \times 2$ experimental matrix was executed. The independent variables were:

- **Server Mode:** ECH enabled (`ech_on`) or ECH disabled (`ech_off`)
- **Client Mode:** ECH-capable client (`ech`) or non-ECH client (`no_ech`)
- **HTTP Mode:** HTTP/1.1 or HTTP/2

This produced eight experimental configurations. Each configuration was executed automatically using the custom `run_matrix.sh` harness.

Server Mode	Client Mode	HTTP Mode	Expected Behaviour
ech_on	ech	http1/http2	ECH Accepted
ech_on	no_ech	http1/http2	ECH Fallback
ech_off	ech	http1/http2	ECH Rejected
ech_off	no_ech	http1/http2	Normal TLS

Table 5.1: Experimental matrix used for ECH behavioural validation

For each run, the following metrics were recorded:

- TLS handshake outcome (accepted, fallback, failed)
- TLS handshake duration (ms)
- HTTP status code
- HTTP protocol negotiation (1.1 or 2)
- Total request time (seconds)

5.2 Results

All eight configurations executed successfully at the HTTP layer. The server started correctly in all cases (0 startup failures). All requests returned HTTP 200, and both HTTP/1.1 and HTTP/2 negotiation succeeded in all runs.

ECH Behavioural Outcomes

- ECH Accepted: 2 runs (ech_on + ech client)
- ECH Fallback: 4 runs (all no_ech client runs)
- ECH Rejected: 2 runs (ech_off + ech client)

The two rejection cases correspond exactly to scenarios where the client attempted ECH while the server had ECH disabled. In these cases, the BoringSSL client reported ECH_REJECTED, confirming correct enforcement of server configuration.

Handshake and Request Timing

Representative measurements (in milliseconds) are shown below:

Configuration	Handshake (ms)	Total Time (s)
ech_on + ech + http1	77	0.0097
ech_on + no_ech + http1	37	0.0104
ech_off + no_ech + http1	31	0.0164

Across all runs:

- HTTP success rate: 8/8 (100%)
- Protocol negotiation success: 8/8 (100%)

- Server stability: 100%

The results confirm correct ECH negotiation behaviour without negatively impacting HTTP functionality.

5.3 Summary

This evaluation demonstrates that the implemented test harness correctly distinguishes between ECH acceptance, fallback, and rejection scenarios across both HTTP/1.1 and HTTP/2.

All configurations executed without server instability, and application-layer communication remained fully functional. The results validate both the correctness of the ECH configuration and the reliability of the automated evaluation framework.

These findings establish a stable foundation for subsequent performance analysis and further experimentation involving real-world discovery mechanisms or browser-based clients.

Chapter 6

Conclusions & Future Work

This chapter should summarize the work presented in the dissertation and discuss the conclusions that can be drawn from the work and the results presented in chapter 5.

6.1 Future Work

The section may present a list of items that were beyond the scope of the dissertation.

Bibliography